

CERTIFICACIÓN UNIVERSITARIA



UNC



FCEyN



Manual para el
alumno

Docker I



WE WANT
SKILLS!

mundosE

Manual para el alumno: Introducción a Docker I.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Contenedores y virtualización ligera.....	3
1.1 Beneficios principales.....	3
1.2 Limitaciones y realidades.....	4
1.3 Casos de uso reales.....	4
2. ¿Qué es Docker? Ventajas y arquitectura.....	5
2.1 Componentes clave.....	5
2.2 Arquitectura en capas.....	6
2.3 Ventajas principales.....	6
2.4 Mitos comunes.....	6
2.5 Flujo pull→run→push.....	7
3. Instalación y configuración.....	7
3.1 Requisitos y recomendaciones.....	7
3.2 Instalación paso a paso (Ubuntu).....	8
3.3 Chequeos iniciales.....	9
3.4 Buenas prácticas de configuración.....	9
3.5 Docker Desktop (Windows/macOS) highlights.....	10
4. Comandos básicos de Docker.....	10
4.1 Ejemplo de ciclo de vida completo.....	11
4.2 Buenas prácticas.....	12
4.3 Anti-patronos frecuentes.....	12
5. Docker Hub y ejecución de imágenes.....	13
5.1 Sintaxis de nombres.....	13
5.2 Búsqueda, descarga y ejecución.....	13
5.3 Subir tu propia imagen.....	14
5.4 Buenas prácticas antes de pull.....	14
5.5 Gobernanza corporativa.....	15
Bibliografía.....	16

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este encontraremos un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: Introducción a Docker I

Objetivo del encuentro:

Reconocer los fundamentos de Docker y comprender cómo los contenedores simplifican la creación, el despliegue y la administración de aplicaciones en entornos aislados. Al finalizar este módulo, podrás:

- Explicar la diferencia entre contenedores y máquinas virtuales.
- Instalar Docker en su propio equipo.
- Utilizar los comandos básicos de la CLI.
- Descargar y ejecutar imágenes públicas desde Docker Hub.
- Identificar buenas prácticas iniciales de seguridad y optimización.

Contenido a desarrollar:

- ✓ Contenedores y virtualización ligera
- ✓ ¿Qué es Docker? Ventajas y arquitectura
- ✓ Instalación y configuración
- ✓ Comandos básicos de Docker
- ✓ Docker Hub y ejecución de imágenes

1. Contenedores y virtualización ligera

Durante muchos años el aislamiento de aplicaciones se resolvió con **máquinas virtuales (VMs)**. Una VM emula *hardware* completo a través de un hipervisor (KVM, VMware, Hyper-V, VirtualBox). Este enfoque brinda un aislamiento muy robusto —cada VM corre su propio kernel y stack de drivers— pero introduce sobrecarga de CPU, memoria y disco. Por ejemplo, una VM de Ubuntu “liviana” suele ocupar entre 8 y 10 GB en disco, arrancar en más de un minuto y reservar unos 700 MB de RAM en reposo. Con la llegada de los microservicios, la necesidad de levantar decenas o cientos de entornos por aplicación volvió a esta sobrecarga un cuello de botella económico y operativo.

Ahí aparecen los **contenedores**, implementados gracias a dos tecnologías del *kernel* Linux que existían desde 2006, pero no eran sencillas de usar: **namespaces** y **cgroups**. Los *namespaces* dividen grandes recursos globales en instancias aisladas —PID, red, sistema de archivos, usuarios—, mientras que los *cgroups* limitan y contabilizan CPU, memoria, E/S y dispositivos para cada grupo de procesos. El resultado es un entorno que *comparte* el kernel del host, pero le hace creer al proceso que vive en su propio sistema.

La analogía clásica es **casa vs. departamento**:

- **Casa (VM):** cada familia levanta paredes, instala su sistema eléctrico y de plomería; la independencia es total, pero el costo de materiales y tiempo también.
- **Departamento (contenedor):** las familias comparten la estructura del edificio y los servicios centrales (kernel), cada unidad conserva privacidad y personaliza su espacio. El costo marginal de sumar otro departamento es mucho menor.

1.1 Beneficios principales

1. **Arranque ultrarrápido.** Un contenedor típico inicia en segundos, a veces milisegundos, porque no *bootea kernel* ni servicios de sistema; simplemente lanza el proceso principal.
2. **Densidad.** En un servidor con 8 GB de RAM puedes ejecutar decenas de contenedores Node.js o Python, mientras que, tal vez, solo cuatro VMs decentes.
3. **Portabilidad.** La imagen empaqueta *runtime* y dependencias, por lo que “funciona en mi máquina” deja de ser un problema: lo que pruebas localmente correrá igual en producción.

4. **Escalabilidad elástica.** Al ser livianos, replicar un contenedor bajo demanda es rápido y barato.
5. **Dev & Ops integrados.** Los equipos DevOps definen la imagen en un **Dockerfile**. Ese artefacto cruza compilación, pruebas y producción sin cambios.

1.2 Limitaciones y realidades

- **Compartir *kernel* implica ciertos riesgos.** Existe la posibilidad de un *container breakout* si hay vulnerabilidades de *kernel*. Por ello, se recomiendan buenas prácticas como **root-less containers**, perfiles AppArmor o SELinux y actualización frecuente.
- **Diferencia de *kernel*.** Si necesitas probar módulos o *drivers* en un *kernel* distinto al del *host*, la VM sigue siendo la opción.
- **Windows containers.** Docker también soporta contenedores Windows, pero son conceptualmente diferentes: requieren un *kernel* Windows y no son portables a Linux.

1.3 Casos de uso reales

- **CI/CD:** GitLab CI y GitHub Actions usan contenedores efímeros para *runners*. Cada *build* recibe un entorno limpio que se descarta al concluir.
- **Microservicios:** *fintechs* argentinas como Ualá despliegan cientos de contenedores por servicio, garantizando reproducibilidad y escalabilidad horizontal.
- **Data Science:** *notebooks* Jupyter empaquetados en imágenes permiten a investigadores compartir exactamente el mismo entorno.

Recursos para profundizar

- **Merkel, D.** (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*.
<https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>
- **Ovens, S.** (2021). Building a Linux container by hand using namespaces. *Red Hat Blog*. <https://www.redhat.com/en/blog/building-container-namespaces>
- **Ovens, S.** (2021). The 7 most used Linux namespaces. *Red Hat Blog*.

- <https://www.redhat.com/en/blog/7-linux-namespaces>
- **Play with Docker** © 2017 - 2023. Playground interactivo *Play-with-Docker*: <https://labs.play-with-docker.com/>
- **Red Hat** ©. (2025). <https://developers.redhat.com/>
- **Red Hat** ©. (2025). Comprensión de los contenedores de Linux. <https://www.redhat.com/en#containers>
- **Tejun Heo** (2015) Cgroups v2 Documentation. *kernel.org*, <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>

2. ¿Qué es Docker? Ventajas y arquitectura

Docker es la plataforma de código abierto que democratizó los contenedores al proporcionar una experiencia de uso sencilla, un formato de imagen estandarizado y un ecosistema robusto. Lanzado en 2013 por Solomon Hykes, Docker agregó una capa de usabilidad sobre LXC y convirtió una herramienta de nicho en un componente central de la moderna ingeniería de *software*.

2.1 Componentes clave

- A. **Docker Engine** – el daemon **dockerd**.
 - a. Ejecuta contenedores, gestiona redes, volúmenes e imágenes.
 - b. Expone una API REST (socket Unix o TCP 2375 -sin TLS- / 2376 -con TLS-).

Docker CLI – binario **docker** que traduce comandos en llamadas a la API.
Ejemplo:

```
docker run nginx
```

- B. Envía un JSON al daemon para descargar la imagen (si falta), crear la capa de escritura y arrancar el contenedor.
- C. **Docker Registry** – repositorio de imágenes versionadas. El predeterminado es **Docker Hub** (**docker.io**), pero existen alternativas: GitHub Container Registry, GitLab Registry, Amazon ECR, Google Artifact Registry, Harbor on-premise.

Un nombre completo de imagen sigue la forma **[REGISTRY/]USUARIO/REPO[:TAG]**. Si omite **REGISTRY**, la CLI asume Docker Hub; si omite **TAG**, asume **latest**.

2.2 Arquitectura en capas

Cada instrucción del **Dockerfile** (**RUN**, **COPY**, **ADD**) crea una **capa solo lectura** identificada por un SHA-256. Cuando se instancian, Docker añade una capa de escritura *copy-on-write* encima. Este esquema permite:

- Cachear builds (si las capas anteriores no cambian, se reutilizan).
- Descargar y subir solo deltas, ahorrando ancho de banda.
- Ver versiones inmutables; puedes fijar un digest (**@sha256:...**) para congelar una imagen.

2.3 Ventajas principales

1. **Velocidad.** Reducir el ciclo “dev → prod” de horas a minutos.
2. **Reproducibilidad.** El mismo contenedor corre igual en tu *notebook*, en un *servidor on-prem* o en la nube.
3. **Portabilidad.** Formato de imagen estándar **OCI** (Open Container Initiative).
4. **Integración.** Amplio soporte en Jenkins, GitLab CI, Kubernetes, VS Code, IntelliJ.
5. **Ecosistema.** Herramientas como Docker Compose (orquestración simple), Docker Swarm (clúster nativo), extensiones de VS Code, plugins de volúmenes/redes, escáneres de seguridad Trivy y Snyk.

2.4 Mitos comunes

- “*Docker es lo mismo que Kubernetes*”: Kubernetes **orquesta** contenedores (pod scheduling, escalado, auto-heal). Docker los construye y ejecuta.
- “*Docker mata a las VMs*”: en realidad se complementan. Muchas empresas corren clústeres Docker dentro de VMs para aislar *tenants* y aprovechar *snapshots*.
- “*Docker es inseguro*”: mal configurado, cualquier *software* lo es. Con *root-less mode*, SELinux, AppArmor y actualizaciones regulares, los riesgos se minimizan.

2.5 Flujo pull→run→push

1. **pull**: la CLI consulta el digest de la imagen; descarga solo las capas faltantes.
2. **run**: el daemon crea un contenedor (capa RW) y lanza el proceso PID 1.
3. **push**: al subir tu propia imagen, el daemon envía únicamente las capas nuevas al registry.

Recursos para profundizar

- **Docker** © 2013-2025. Docker CLI Reference: <https://docs.docker.com/engine/reference/commandline/cli/>
- **Docker** © 2013-2025. Docker Engine Overview: <https://docs.docker.com/engine/>
- **OCI Image Spec v1.0.2**: <https://github.com/opencontainers/image-spec/tree/v1.0.2>
- **Poulton, N.** Capítulo 2 “Core Docker Components”. En **Docker Deep Dive**. <https://nigelpoulton.com/books/docker-deep-dive/>
- **Rice, L.** [GOTO Conferences] (2018, jun 29). Containers From Scratch • Liz Rice • GOTO 2018: <https://www.youtube.com/watch?v=8fi7uSYIOdc>

3. Instalación y configuración

Existen dos grandes formas de instalar Docker: **Docker Desktop** (Windows / macOS) y **Docker Engine** nativo (GNU/Linux). Docker Desktop empaqueta la CLI, el Engine dentro de una VM ligera, Kubernetes opcional y herramientas gráficas; está pensado para entornos de desarrollo. En servidores Linux productivos se suele instalar solo el Engine para maximizar recursos.

3.1 Requisitos y recomendaciones

Sistema	Requisitos mínimos	Observaciones
Ubuntu 22.04+	kernel $\geq$ 3.10, cgroups v2, overlayFS	Motor nativo, mejor rendimiento.
CentOS Stream 9	SELinux en modo <i>permissive</i> o perfiles tuned	DNF repo oficial.

Windows 10 Pro/11 WSL2 o Hyper-V habilitado
Pro

Docker Desktop instala
WSL kernel propio.

macOS 12+ Intel VT-x o Apple Silicon

Desktop usa Apple HVF
para virtualizar

3.2 Instalación paso a paso (Ubuntu)

Copia el bloque completo:

```
# 1. Paquetes previos
sudo apt update && sudo apt install -y ca-certificates curl gnupg
lsb-release

# 2. Clave GPG
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/docker.gpg

# 3. Repo estable
echo \
  "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(lsb_release -cs)
stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# 4. Instalación
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io

# 5. Habilitar y arrancar servicio
sudo systemctl enable --now docker

# 6. Agregar usuario al grupo docker
sudo usermod -aG docker $USER
newgrp docker

# 7. Prueba
docker run hello-world
```

3.3 Chequeos iniciales

```
docker --version          # muestra versión
docker info | less        # detalles de almacenamiento, cgroups,
drivers
docker system df          # espacio ocupado
```

Si `docker run hello-world` imprime *"Hello from Docker!"*, la instalación fue exitosa.

3.4 Buenas prácticas de configuración

Root-less mode. Ejecuta:

```
dockerd-rootless-setuptool.sh install
```

- El daemon se lanza como usuario sin privilegios, aislando `/run/user/$UID/docker`.

Logging eficiente. Cambia el driver por defecto a `local` y limita tamaño:

```
json
CopyEdit
{
  "log-driver": "local",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

- Guarda en `/etc/docker/daemon.json` y reinicia.

Limpieza programada. Agrega en tu `crontab`:

```
0 3 * * 0 docker system prune -af --volumes
```

- Así evitas que `/var/lib/docker` crezca sin control.
- **TLS para acceso remoto.** Solo expón el daemon sobre 2376 con certificados; nunca sobre 2375 sin cifrar.

3.5 Docker Desktop (Windows/macOS) highlights

- En **Settings** → **Resources**, limita CPU y RAM para no saturar la *notebook*.
- Activa **Kubernetes** si quieres practicar orquestación local.
- Apple Silicon: marca “*Use Rosetta for x86/amd64 emulation*” para ejecutar imágenes Intel.

Recursos para profundizar

- **Docker** © 2013-2025. *Configure and troubleshoot the Docker daemon*:
<https://docs.docker.com/config/daemon/>
- **Docker** © 2013-2025. *Docker Desktop documentation*:
<https://docs.docker.com/desktop>
- **Docker** © 2013-2025. *Install Docker Engine on Ubuntu*.
<https://docs.docker.com/engine/install/ubuntu/>
- **Docker** © 2013-2025. *Rootless mode* – Docker docs:
<https://docs.docker.com/engine/security/rootless/>
- **Microsoft** © 2025. Tutorial “Running Docker on WSL 2” – Microsoft Learn:
<https://learn.microsoft.com/windows/wsl/tutorials/wsl-containers>

4. Comandos básicos de Docker

El trabajo diario con Docker se resume en **crear**, **listar**, **inspeccionar**, **detener** y **eliminar** contenedores, además de gestionar imágenes. A continuación, tienes una guía exhaustiva de los comandos elementales, sus *flags* más útiles y las buenas prácticas asociadas.

Comando	Descripción conceptual	Flags esenciales
<code>docker run</code>	Crea y arranca un contenedor; combina <code>pull</code> , <code>create</code> , <code>start</code> .	<code>-d</code> (detached), <code>-it</code> , <code>--name</code> , <code>-p host:cont</code> , <code>--rm</code> , <code>-e</code>
<code>docker ps</code>	Lista de contenedores en ejecución.	<code>-a</code> incluye detenidos, <code>--format</code> personaliza salida
<code>docker stop</code>	Envía SIGTERM y espera 10 s antes de SIGKILL.	<code>-t segundos</code>
<code>docker kill</code>	Envía SIGKILL inmediatamente (último recurso).	—

<code>docker rm</code>	Elimina contenedores detenidos.	<code>-f</code> fuerza, <code>-v</code> borra volúmenes anónimos
<code>docker images</code>	Muestra imágenes locales.	<code>--digests</code>
<code>docker pull</code>	Descarga una imagen del registry.	<code>--platform linux/arm64</code>
<code>docker rmi</code>	Borra imágenes (si no hay contenedores dependientes).	<code>-f</code>
<code>docker logs</code>	Lee stdout/stderr del contenedor.	<code>-f, --since, --tail</code>
<code>docker exec</code>	Ejecuta comando dentro de un contenedor activo.	<code>-it</code>
<code>docker system df</code>	Reporta uso de disco por Docker.	<code>—</code>
<code>docker system prune</code>	Limpia recursos “huérfanos” (dangling).	<code>-af, --volumes</code>

4.1 Ejemplo de ciclo de vida completo

1. Crear y arrancar

```
docker run -d --name web -p 8080:80 nginx:alpine
```

2. Ver estado

```
docker ps
```

```
docker logs -f web
```

3. Ingresar al contenedor

```
docker exec -it web /bin/sh
```

4. Detener y eliminar

```
docker stop web
```

```
docker rm web
```

5. Limpiar la imagen

```
docker rmi nginx:alpine
```

4.2 Buenas prácticas

- **Evitar `latest` en producción:** fija versión (`nginx:1.29.0-alpine`) o digest para reproducibilidad.
- **Nomenclatura coherente:** siempre asigna `--name`, por ejemplo `web-prod`, `api-test`. Facilita logs y scripts.
- **Contenedores efímeros:** `--rm` borra el contenedor al salir, ideal para pruebas unitarias.

Time to gracefully stop: si tu aplicación necesita tiempo para cerrar conexiones, aumenta el timeout:

```
docker stop -t 30 web
```

- **Logs:** sigue la regla *“los contenedores solo loguean a stdout/stderr”* y usa herramientas externas (Fluent Bit, Loki, Elasticsearch) para recolectar.
- **No root:** utiliza la instrucción `USER app` en el Dockerfile o ejecuta en root-less mode.

4.3 Anti-patrones frecuentes

- **Ejecutar `docker exec` en producción** para modificar archivos manualmente: rompe la inmutabilidad. Si necesitas cambiar algo, genera una nueva imagen.
- **Montar `/var/run/docker.sock`** en contenedores sin justificarlo: otorga acceso *root* al *host*.
- **Dejar contenedores detenidos:** ocupan disco y complican monitoreo. Límpialos o usa `--rm`.

Al dominar estos comandos, entenderás el flujo de trabajo diario con Docker y podrás automatizarlo en scripts CI.

Recursos para profundizar

- *Docker Command Line Cheat Sheet* (PDF):

- <https://dockerlabs.collabnix.com/docker/cheatsheet/>
- **Docker** © 2013-2025. *Docker run reference*.
<https://docs.docker.com/engine/reference/run/>
- **IBM Training:** Docker Essentials: A Developer Introduction –
<https://www.ibm.com/training/badge/docker-essentials-a-developer-introduction>

5. Docker Hub y ejecución de imágenes

Un **registry** es un repositorio de imágenes versionadas. Docker Hub, operado por Docker Inc., es el *default* cuando no se especifica otro. Allí conviven tres categorías de imágenes:

1. **Official Images** — mantenidas por Docker y el proyecto upstream (ej.: `nginx`, `postgres`).
2. **Verified Publisher** — subidas por empresas que validaron su identidad (ej.: `bitnami/nginx`).
3. **Community** — cualquier usuario; ofrecen flexibilidad pero exigen auditoría.

5.1 Sintaxis de nombres

css

CopyEdit

[REGISTRY/]USUARIO/REPO[:TAG]

- Si falta **REGISTRY**, se asume `docker.io`.
- Si falta **TAG**, se asume `latest`. Esto es cómodo, pero peligroso: la referencia puede cambiar sin aviso. Opta por *tags* semánticos (`1.0.2`) o *digest*:
`graphql`

`redis@sha256:6b98df3e...`

5.2 Búsqueda, descarga y ejecución

Buscar imágenes populares

```
docker search redis --filter stars=5000
```

```
# Descargar una versión concreta
```

```
docker pull redis:7.2-alpine
```

```
# Instanciarla
```

```
docker run -d --name cache -p 6379:6379 redis:7.2-alpine
```

Docker descarga solo las capas que no existen localmente. Si tu contenedor necesita arquitectura ARM, usa `--platform linux/arm64`.

5.3 Subir tu propia imagen

```
docker tag hola:v1 pabloalumno/hola:v1
```

```
docker login -u pabloalumno
```

```
docker push pabloalumno/hola:v1
```

Para proyectos corporativos conviene un registro privado (Harbor, ECR) que ofrezca control de acceso, escaneo automático y almacenamiento on-prem o en la nube local.

5.4 Buenas prácticas antes de *pull*

1. **Procedencia:** preferí Oficial / Verified; si es Community, revisa el Dockerfile.
2. **Tamaño:** imágenes Alpine (< 100 MB) vs. Ubuntu (~ 200 MB) vs. distroless (< 50 MB).

Seguridad: escanea cada imagen. Ejemplo con **Trivy**:

```
trivy image redis:7.2-alpine
```

3. **Licenciamiento:** asegúrate de que las librerías incluidas sean compatibles con tu modelo de negocio.
4. **Rate limits:** cuentas gratuitas anónimas tienen 100 pulls/6 h; autenticadas, 200 pulls/6 h. Implementa un **pull-through cache** para CI intensivo.
5. **Firmas:** usa **cosign** o Notary v2 para firmar y verificar que la imagen no fue modificada.

5.5 Gobernanza corporativa

Empresas maduras establecen un **catálogo interno** de imágenes aprobadas. Cualquier *pull* externo pasa primero por un proxy que las escanea. Así evitan CVEs y licencias prohibidas. Herramientas populares: Harbor + Clair, JFrog Xray, Snyk Container.

Recursos para profundizar

- **Docker** © 2013-2025. *Docker Hub Quickstart*. <https://docs.docker.com/docker-hub/>
- **Docker** © 2013-2025. *Pull rate limits FAQ*: <https://docs.docker.com/docker-hub/download-rate-limit/>
- *Cosign User Guide* – sigstore: <https://docs.sigstore.dev/cosign/signing/overview/>
- *Scanner open source Trivy*: <https://aquasecurity.github.io/trivy/>

Bibliografía

- **Aqua Security Blog.** “Best Practices for Container Image Scanning”, 2024.
- **Docker** © 2013-2025. Docker Documentation – <https://docs.docker.com/> (consultado julio, 2025).
- **Docker** © 2013-2025. Docker Hub Docs – <https://docs.docker.com/docker-hub/> (consultado julio, 2025).
- **Merkel, D.** (2014). Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux Journal*.
- **OCI Image Spec v1.0.2** – <https://opencontainers.org/>
- **Poulton, N.** (2023). *Docker Deep Dive*, 5.^a ed., 2023.
- **Snyk Ltd.** (2023). Container Security Guide.
- **Turnbull, J.** (2022). The Docker Book, 5.^a ed., 2022.